

Containers and Containerisation

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departamento de Engenharia Informática

Containerisation Fundamentals

(Recalling) The Evolution of Server Computation

- **Bare-Metal Deployments**

- ▶ Direct execution on physical hardware.
- ▶ Excellent performance, but poor resource utilization and slow provisioning.

- **Hardware Virtualisation (VMs)**

- ▶ Managed by a Hypervisor (Type 1 or Type 2).
- ▶ Complete hardware abstraction; each virtual machine runs a full Guest OS.

- **Operating System Virtualisation (Containers)**

- ▶ Abstraction at the OS user-space level.
- ▶ Applications share the host kernel while maintaining isolated environments.

Demystifying Containers

- **The Myth:** "A container is just a lightweight Virtual Machine."
- **The Technical Reality:** A container is simply an isolated Linux process running natively on the host kernel.
- **No Guest OS:** Eliminates hypervisor overhead and redundant OS kernels.
- **Near-Instant Start Times:** Boots as fast as starting a standard process.
- **High Density:** Significantly higher container-to-host density compared to VMs.

Linux Namespaces: Creating the Illusion of Isolation

Namespaces restrict **what a process can see** by wrapping global system resources in isolated abstractions.

PID Process IDs — Isolates process trees (container process sees itself as PID 1).

NET Network interfaces, IP routing tables, port bindings, and firewall rules.

MNT Mount points — Provides an isolated file system view.

IPC Inter-Process Communication (POSIX/System V message queues, shared memory).

UTS Hostname and NIS domain name isolation.

USER Maps container root/user UIDs to unprivileged UIDs on the host.

Control Groups (cgroups): Resource Allocation

While Namespaces control *visibility*, Control Groups govern *resource consumption*.

- **Core Responsibilities:**

- ▶ **Resource Limiting:** Cap maximum memory usage, CPU shares, and disk I/O.
- ▶ **Prioritisation:** Allocate relative CPU time bands to competing workloads.
- ▶ **Accounting:** Measure and audit resource utilization per group.
- ▶ **Control:** Freeze, resume, or terminate groups of processes safely.

- **Preventing the "Noisy Neighbour":**

- ▶ Restricts run-away container processes from consuming host resources and crashing adjacent workloads.

Container Security Principles: Restricting Privilege

Kernel sharing requires strict boundary controls to prevent privilege escalation.

- **The Root Problem in Containers**

- ▶ The root user inside an unconstrained container matches UID 0 on the host kernel.
- ▶ Full superuser access would allow a compromised container to manipulate host hardware, devices, and kernel memory.

- **Defence-in-Depth Mechanisms**

- ▶ **Linux Capabilities:** Fine-grained administrative permissions.
- ▶ **Seccomp Profiles:** System call filtering at the kernel boundary.
- ▶ **Read-Only Filesystems & User Namespaces:** Additional containment layers.

Linux Capabilities: Granular Control

Linux splits traditional monolithic root privileges into distinct, unit-level capabilities.

- **Key Capabilities Examples**

- ▶ CAP_NET_ADMIN: Configure network interfaces, routing tables, and firewall rules.
- ▶ CAP_SYS_ADMIN: Perform system administration tasks (e.g., mounting filesystems).
- ▶ CAP_CHOWN: Change file ownership settings.

- **Container Engine Defaults**

- ▶ Container runtimes drop dangerous capabilities by default (e.g., CAP_SYS_ADMIN).
- ▶ **Best Practice:** Operate on the principle of least privilege:
 - ▶ Drop all capabilities: `-cap-drop=ALL`
 - ▶ Selectively re-add required capabilities: `-cap-add=NET_BIND_SERVICE`

Seccomp: System Call Filtering

Secure Computing Mode (seccomp) restricts the system calls (*syscalls*) a process can issue to the host kernel.

- **Mechanism**

- ▶ Intercepts syscalls directly before kernel execution.
- ▶ Evaluates requests against a JSON-defined whitelist profile.

- **Default Protection**

- ▶ Out of over 300+ Linux syscalls, standard profiles block around 40+ high-risk calls by default (e.g., reboot, kexec_load, sys_ptrace).

- **Security Impact**

- ▶ Blocks dangerous kernel interactions even if a process gains root execution within the container environment.

Container Philosophies: System vs Application

Not all container technologies serve the same architectural purpose. They generally fall into two distinct design philosophies:

- **System Containers (LXC/LXD)**

- ▶ Modelled after traditional virtual machines.
- ▶ Focuses on providing a multi-process, long-lived Linux environment.

- **Application Containers (Docker/OCI)**

- ▶ Modelled around software packaging and distribution.
- ▶ Focuses on isolating and running a single application process.

System containers virtualise the *operating system environment*, whereas application containers package the *application and its dependencies*.

System Containers

Linux Containers (LXC) and Linux Container Daemons (LXD)

- **Architectural Approach**

- ▶ Operates as a full lightweight Linux system sharing the host kernel.
- ▶ Boots a complete initialisation system (systemd, init).

- **Operational Characteristics**

- ▶ Runs multiple background services, daemons, and SSH in a single container.
- ▶ Mutable state: updated, patched, and configured like a traditional OS or VM.
- ▶ Persistent lifecycle: designed to run continuously over long periods.

- **Primary Use Cases**

- ▶ Multi-tenant infrastructure host environments.
- ▶ Legacy application migration where splitting services is impractical.

Application Containers

Docker and the Open Container Initiative (OCI)

- **Architectural Approach**

- ▶ Designed to execute a single main foreground process (PID 1).
- ▶ Bypasses traditional OS init systems entirely.

- **Operational Characteristics**

- ▶ **Stateless and Ephemeral:** Discarded and recreated rather than updated in-place.
- ▶ **Immutable Infrastructure:** Built from read-only image layers distributed via registries.
- ▶ **Declarative Build Process:** Defined reproducibly using standard build scripts (Dockerfile).

- **Primary Use Cases**

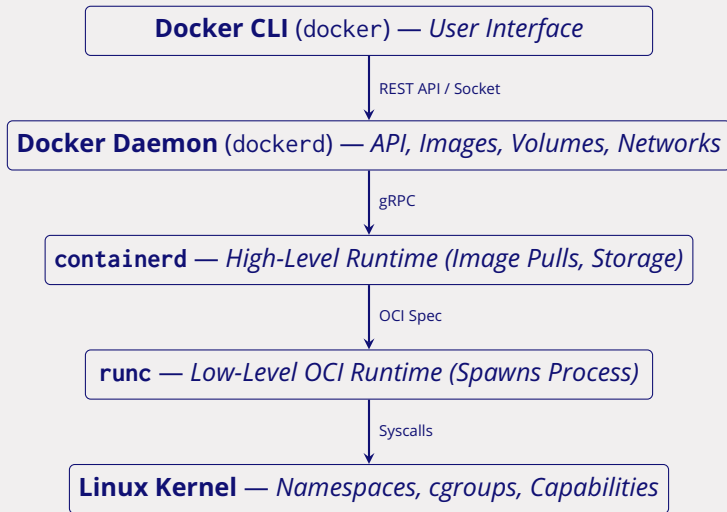
- ▶ Microservices architectures and cloud-native applications.
- ▶ CI/CD pipelines, automated testing, and horizontal scaling.

Docker Architecture and Image Anatomy

Docker Engine Architecture

The Docker platform is composed of decoupled, modular components operating at different abstraction layers.

- **Docker CLI (docker)**
 - ▶ The command-line client used by developers to issue high-level commands.
- **Docker Daemon (dockerd)**
 - ▶ The background service managing images, networks, volumes, and API requests.
- **containerd**
 - ▶ An industry-standard, high-level container runtime that manages image transfer, execution, and storage lifecycle.
- **runc**
 - ▶ A lightweight, low-level OCI-compliant runtime that interacts directly with kernel namespaces and cgroups to spawn containers.



Practical Example: Running a Container

1. **Command Execution:** User enters `docker run -d nginx` in the CLI.
2. **API Request:** The CLI translates the command into a REST API payload sent via Unix Domain Socket to `dockerd`.
3. **Image Resolution:** `dockerd` queries `containerd` to check local image availability; if missing, it fetches layers from the remote registry.
4. **Runtime Hand-off:** `containerd` unpacks OCI layers and delegates execution to `runc`.
5. **Kernel Execution:** `runc` configures namespaces, cgroups, and capabilities, starts PID 1, and exits, leaving the container process running.

Image Anatomy: Layers and Union File Systems

Docker images are ordered collections of read-only file system modifications.

- **Union File System (UnionFS)**

- ▶ Combines separate directories into a single virtual file system view.
- ▶ **Overlay2** is the default storage driver in modern Linux distributions.

- **Layer Immutability**

- ▶ Each instruction in a Dockerfile creates an immutable, content-addressable layer identified by a SHA256 hash.

- **Block Sharing and Efficiency**

- ▶ Identical layers are stored once on host disk and shared across multiple images and running containers.

Internal Anatomy: Image Layer Stacking

How build instructions construct a unified file system view:

- **Layer Composition Pipeline**

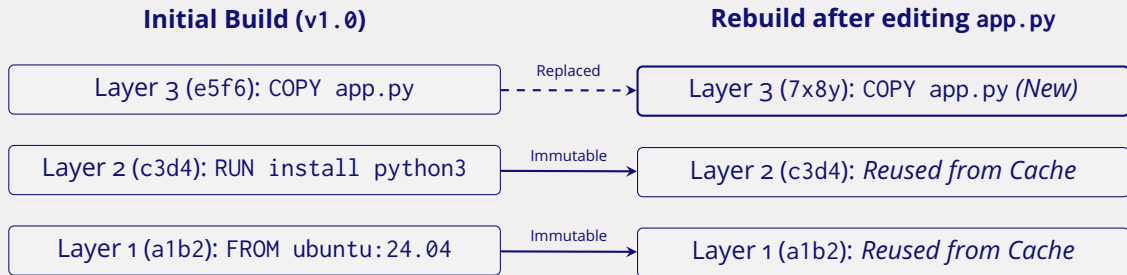
- ▶ FROM ubuntu:24.04 → Base OS layer (Read-Only)
Contains distribution root filesystem binaries and C-libraries (glibc). Does not include a kernel or bootloader.
- ▶ RUN apt-get update && apt-get install -y python3 → Dependency layer (Read-Only)
- ▶ COPY app.py /app/ → Application code layer (Read-Only)

- **Overlay2 View Aggregation**

- ▶ **LowerDir:** Read-only image layers stacked sequentially.
- ▶ **MergedDir:** Unified file system interface visible to the process inside the container.

Visualising Immutability: The Layer Stack

Immutability means previous layers are locked and read-only. Modifying files creates new layers instead of altering existing ones.



Container State: Read-Only vs. Read-Write Layer

Understanding state management during container runtime.

- **Container Layer (Read-Write)**

- ▶ When a container starts, a thin, mutable Read-Write layer is added on top of the image stack.

- **Copy-on-Write (CoW) Strategy**

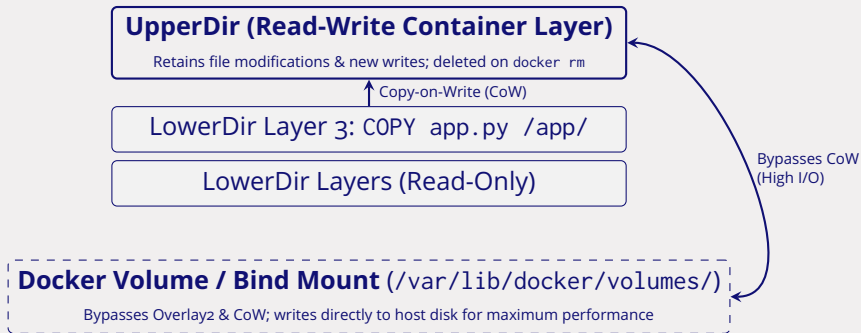
- ▶ Modifying an existing file copies it from the read-only LowerDir up to the UpperDir (R/W layer) before editing.

- **Performance Implications**

- ▶ CoW operations introduce I/O latency for heavy write workloads.
- ▶ **Best Practice:** Use Docker Volumes or Bind Mounts for database files and persistent state.

Container Storage Architecture: CoW vs Volumes

Unified Container File System View (MergedDir)



Comparing Base Images (generated by Gemini, Google)

Selecting an appropriate base image balances size, runtime dependencies, and attack surface.

Base Image	Approx. Size	Ecosystem	Typical Use Case
ubuntu:24.04	~75 MB	Debian/APT	Full-featured CLI tooling, legacy compatibility.
node:20-alpine	~180 MB	Alpine/APK	JavaScript applications with pre-installed Node.js.
alpine:3.19	~5 MB	Alpine/APK	Minimalist base with package manager for light builds.
scratch	0 MB	None	Compiled static binaries (e.g. Go, Rust) with zero overhead.

Image Distribution: Registries and Tags

Distributing OCI-compliant artifacts across environments.

- **Registries and Repositories**

- ▶ Registries (e.g. Docker Hub, GHCR, ECR) host image repositories.
- ▶ Built on the Open Container Initiative (OCI) Distribution Specification.

- **Tagging Strategies**

- ▶ Tags are pointers to specific layer manifest hashes.
- ▶ Mutable tags (e.g. :latest) change when updated images are pushed.

- **Production Best Practices**

- ▶ Avoid using :latest in production environments due to unpredictability.
- ▶ Use semantic versioning (e.g. node:20.11-alpine) or pin immutable SHA256 digests.

Dockerfiles

Anatomy and Best Practices for Building Container Images

The Recipe for Image Creation

A Dockerfile is a declarative script containing step-by-step instructions to build an OCI-compliant image.

- **Declarative Build Definition**

- ▶ Replaces manual container modification with reproducible, version-controlled code.

- **Syntax and Conventions**

- ▶ Instructions are written in uppercase by convention (e.g. FROM, RUN, COPY).
- ▶ Evaluated sequentially from top to bottom by the Docker daemon build context.

- **Caching Mechanics**

- ▶ Each instruction creates a distinct layer.
- ▶ If an instruction and its inputs remain unchanged, Docker reuses the cached layer from previous builds.

Fundamental Instructions: Origin, Space, and Execution (1/2)

Core directives forming the baseline of every container image definition.

- **FROM**

- ▶ Initializes a new build stage and sets the user-space base image (LowerDir).
- ▶ Must be the first non-comment instruction in a Dockerfile.

- **WORKDIR**

- ▶ Sets the working directory for subsequent RUN, CMD, ENTRYPOINT, COPY, and ADD instructions.
- ▶ Automatically creates directory paths if they do not exist.

Fundamental Instructions: Origin, Space, and Execution (2/2)

Core directives forming the baseline of every container image definition.

- **RUN**
 - ▶ Executes commands in a new layer on top of the current image and commits the results.
- **EXPOSE**
 - ▶ Documents intended listening ports at runtime (acts as metadata; does not publish ports automatically).

Practical Example

An annotated Dockerfile for a lightweight Python web application.

```
# Step 1: Set user-space base image
FROM python:3.11-slim

# Step 2: Set working directory inside the container
WORKDIR /app

# Step 3: Copy dependency manifests first (optimises build cache)
COPY requirements.txt .

# Step 4: Install runtime dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Step 5: Copy application source code
COPY . .

# Step 6: Document application port
EXPOSE 5000

# Step 7: Define startup command
CMD ["python", "app.py"]
```

File Transfer: COPY vs ADD

Understanding the distinct mechanisms for introducing files into an image layer.

- **COPY (Recommended Default)**

- ▶ Explicitly copies local files or directories from the host build context to the container file system.
- ▶ Transparent, predictable, and devoid of hidden side effects.

- **ADD (Specialised Directive)**

- ▶ Extends COPY with two additional automatic features:
 1. Fetches files directly from remote URLs.
 2. Automatically unpacks **local** compressed archives (.tar, .tar.gz, .tgz) into the target directory.

- **Best Practice Rule**

- ▶ Always use COPY unless local tarball auto-extraction is explicitly required.

Practical Example: File Transfer Pitfalls

Anti-Pattern: Indiscriminate ADD Usage

```
# Fetches archive via remote URL (creates un-cached layer, cannot cleanup)
ADD https://example.com/legacy-app.tar.gz /app/

# Uses ADD for standard file copying (obscures intent)
ADD app.py /app/
```

Practical Example: File Transfer Good Practices

Best Practice: Explicit COPY and RUN Pipeline

```
# Explicitly copy local tarball for automatic extraction
```

```
ADD local-assets.tar.gz /app/assets/
```

```
# Use COPY for explicit application source code transfer
```

```
COPY src/ /app/src/
```

```
# Use curl/wget in RUN to fetch, unpack, and clean up in a single layer
```

```
RUN curl -fsSL https://example.com/data.tar.gz | tar -xz -C /app/data
```

Dynamic Configuration: ARG vs ENV

Managing parameters across build time and runtime execution stages.

- **ARG (Build-Time Variables)**

- ▶ Variables available only during the image build process (`docker build`).
- ▶ Passed via `-build-arg key=value`.
- ▶ **Security Note:** Not persisted in running containers, but visible in image history layer metadata.

- **ENV (Runtime Environment Variables)**

- ▶ Variables defined inside the image and persisted across container execution.
- ▶ Can be overridden at runtime via `docker run -e KEY=VALUE`.

Practical Example: Parameter Injection

```
FROM node:20-alpine

# Build-time parameter with default value
ARG NODE_ENV_BUILD=production
ARG APP_VERSION=1.0.0
# Persist version as environment variable inside image
ENV APP_VERSION=${APP_VERSION}
ENV PORT=8080
WORKDIR /usr/src/app
COPY package*.json ./
# Use ARG during build processing
RUN npm ci --only=${NODE_ENV_BUILD}
COPY . .
EXPOSE ${PORT}
CMD ["node", "server.js"]
```

Practical Example: Parameter Injection

CLI Execution Commands

```
$ docker build --build-arg APP\_VERSION=2.1.0 -t myapp:2.1.0 .
```

```
$ docker run -d -p 8080:8080 -e PORT=8080 myapp:2.1.0
```

Application Entry Point: CMD vs ENTRYPOINT

Defining how a container initializes its primary process (PID 1).

- **CMD (Default Arguments / Commands)**
 - ▶ Specifies default commands or parameters.
 - ▶ Easily overridden by appending arguments to `docker run` (e.g. `docker run myimage bash`).
- **ENTRYPOINT (Fixed Executable)**
 - ▶ Configures a container to run as a dedicated binary CLI tool.
 - ▶ Command-line arguments passed to `docker run` are appended directly to the executable rather than replacing it.

Practical Example: Exec Syntax vs Shell Syntax

Combining ENTRYPOINT and CMD to build flexible containerized executables.

- **Exec Form (Preferred JSON Array):** ["executable", "param1"]
 - ▶ Spawns binary directly as PID 1; correctly receives Unix signals (SIGTERM, SIGINT).
- **Shell Form (String):** executable param1
 - ▶ Wraps process inside /bin/sh -c; prevents signals from reaching the main process.

Practical Example: Combining Directives

```
# Fixed executable
ENTRYPOINT ["/usr/bin/curl"]

# Default parameters (overridable by user at CLI)
CMD ["--help"]
```

- Running **docker run mycurl** executes:
curl -help
- Running **docker run mycurl https://api.example.com** executes
curl https://api.example.com.

Best Practice, Optimisation and Building

Efficient Structuring and Docker Layer Caching

- **How Layer Caching Works:**

- ▶ Docker builds images layer by layer based on Dockerfile instructions.
- ▶ Each instruction creates an intermediate cached layer.

- **Cache Invalidation Rules:**

- ▶ Docker checks if the command and files involved match the previous build.
- ▶ If a layer changes, **all subsequent layers** lose their cache automatically.
- ▶ Commands like `RUN apt-get update` are not re-evaluated unless the instruction text changes or an earlier layer invalidates the cache.

- **Core Optimization Rule:**

- ▶ Order instructions from **least frequently changed** to **most frequently changed**.

Maximising Cache Usage in Practice

Slow Build (Bad Practice)

```
1 FROM node:18-alpine
2 WORKDIR /app
3
4 # Copy all files first
5 COPY . .
6 # Cache breaks whenever ANY
7 # source code file changes
8 RUN npm install
9 CMD ["node", "server.js"]
```

Fast Build (Optimised)

```
1 FROM node:18-alpine
2 WORKDIR /app
3
4 # Copy dependency files only
5 COPY package*.json ./
6
7 # Cached unless package.json
8 # changes
9 RUN npm install
10 # Copy rest of source code
11 COPY . .
12 CMD ["node", "server.js"]
```

Reducing the Build Context: The Role of .dockerignore

- **What is the Build Context?**

- ▶ The set of files sent from your client to the Docker daemon during `docker build`.
- ▶ Unnecessary files slow down build initiation and bloat layer sizes.

- **Why Use a .dockerignore File?**

- ▶ Excludes heavy or sensitive local directories before sending data to the daemon.
- ▶ Prevents overwriting container dependencies with local builds.

Example .dockerignore file:

```
node_modules
.git
.env
*.log
dist
```

Advanced Build Patterns: Multi-Stage Builds

- **The Problem with Single-Stage Builds:**

- ▶ Production images often retain build tools, compilers, source code, and SDKs.
- ▶ Results in massive image sizes and a larger attack surface.

- **The Multi-Stage Solution:**

- ▶ Use multiple FROM statements within a single Dockerfile.
- ▶ Each FROM instruction can use a different base image and begins a new build stage.
- ▶ Copy **only** the compiled artifacts from one stage to another.

- **Key Benefit:** Keep build dependencies in the build phase and deploy a minimal runtime image.

Multi-Stage Build in Action: Go Application (Gemini, Google)

```
# Stage 1: Build stage (heavyweight image)
FROM golang:1.21 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o myapp .

# Stage 2: Final runtime stage (minimal image)
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/myapp .
EXPOSE 8080
CMD ["/myapp"]
```

Multi-Stage Build Architecture Breakdown (1/2)

Stage 1: Build Phase

```
1 FROM golang:1.21 AS builder
2 WORKDIR /app
3 COPY go.mod go.sum ./
4 RUN go mod download
5 COPY . .
6 RUN CGO_ENABLED=0 GOOS=linux \
7     go build -o myapp .
```

- **Heavy Base Image:** Includes compiler, SDKs, and tooling.
- **Cache Optimisation:** Downloads dependencies before copying source files.
- **Static Binary:** CGO_ENABLED=0 creates a self-contained executable.

Multi-Stage Build Architecture Breakdown (2/2)

Stage 2: Minimal Runtime

```
1 FROM alpine:latest
2 WORKDIR /app
3 COPY --from=builder /app/myapp .
4 EXPOSE 8080
5 CMD ["/myapp"]
```

- **Fresh Minimal Base:** Starts from a tiny Alpine Linux image (~5 MB).
- **Targeted Extraction:** Uses `--from=builder` to copy *only* the compiled binary.
- **Discarded Overhead:** Source code and build tools are left behind.

Why Multi-Stage Builds Reduce Image Size

- **Separation of Concerns:**

- ▶ Compilation environments require heavy SDKs and build tools.
- ▶ Runtime environments only require the final executable artifact.

- **Key Advantages:**

- ▶ **Dramatic Reduction in Size:** Drops footprint from hundreds of megabytes to a minimal runtime.
- ▶ **Enhanced Security:** Smaller attack surface by eliminating compilers, shell utilities, and source code from production.
- ▶ **Faster Deployment:** Reduced network bandwidth consumption during registry pulls.

Why Multi-Stage Builds Reduce Image Size

Impact Comparison

Metric	Single-Stage	Multi-Stage (Alpine)
Image Content	Go SDK + Source + Tools	Compiled Binary + Base OS
Final Size	~800 MB	~15 MB
Attack Surface	High	Minimal

Practical Terminal Commands: Build and Run

- **Build the Image:**

```
docker build -t my-app:1.0 .
```

Compiles the image from the current directory (.) and tags it as my-app:1.0.

- **Run the Container:**

```
docker run -d -p 8080:80 --name web-container my-app:1.0
```

Runs detached (-d), maps host port 8080 to container port 80, and assigns a name.

- **Inspect Logs:**

```
docker logs -f web-container
```

Streams real-time output logs (-f) from the running container.

Security Best Practices: Non-Root Execution

- **The Risk of Running as Root:**

- ▶ By default, containers run as the root user.
- ▶ If an attacker escapes the container, they gain full system control on the host.

- **The Principle of Least Privilege:**

- ▶ Create and switch to a dedicated, unprivileged user before starting the application.

Example: Creating and Using a Non-Root User

```
1 FROM node:18-alpine
2 WORKDIR /app
3
4 # Create a system group and user
5 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
6
7 # Copy application files and set correct permissions
8 COPY --chown=appuser:appgroup . .
9
10 # Switch to the unprivileged user
11 USER appuser
12
13 EXPOSE 3000
14 CMD ["node", "server.js"]
```